Relatório de Modernização

ProjectPricingAI — Sistema de Cotação de Projetos

Atividade Prática: Refatoração de Arquitetura · MBA em Engenharia de Software · Engenharia de Software 2.0

Repositório: github.com/rafaelgils/ProjectPricingAI · branch `refactoring` · Pull Request #11

Autor: Thiago Moura

Conteúdo: Parte 1 — Relatório de Modernização · Parte 2 — Prompts utilizados · Parte 3 — Plano de refatoração

Relatório de Modernização — ProjectPricingAI

> Branch `refactoring` · base: `main` no commit `73ccef1` · escopo: auditoria técnica e modernização, sem novas funcionalidades de negócio. Plano de origem: `plano-de-refatoracao.md` · Prompts: `prompts.md` · Decisões: ADR-011 a ADR-015 em `.ai/architecture.md`.

1. Resumo executivo

O sistema (cotação de projetos em linguagem natural, com agente de LLM e cálculo determinístico) estava funcional e bem testado, mas carregava quatro dívidas estruturais que o impediam de operar como produto SaaS: **atendia uma única empresa**, **não era idempotente**, **não tinha camada de proteção de uso/custo** e **tinha o agente concentrado em uma classe de 374 linhas** sem fronteiras verificáveis entre módulos.

Resultado	Antes	Depois
Testes do backend	375	**447** (+72), todos aprovados
Testes do frontend	22	**24**
Cobertura de linhas do backend	não registrada no baseline	**86 %** (mínimo exigido: 70 %)
Maior arquivo do backend	`AgenteProjetos.cs`, 374 linhas	`KeycloakAdminClient.cs`, 252; `AgenteProjetos.cs`, **146**
Isolamento entre empresas	nenhum (`clienteId` apenas)	filtro de `tenantId` num único ponto (repositório base)
POSTs que chamam o LLM idempotentes	0 de 2	**2 de 2** (opcional via `Idempotency-Key`)
Limites de uso	nenhum	3 níveis (IP no Kong; usuário e tenant no backend)
Fronteiras entre camadas/módulos	convenção	**10 testes de arquitetura**
Pacote principal do frontend	566 kB	**424 kB** (páginas em *lazy loading*)
Vulnerabilidades de dependências	0	0

Os 375 testes originais continuam passando. As únicas alterações em testes existentes foram de **assinatura** (construtor dos repositórios, do serviço de usuários e do agente) e a inclusão do `tenantId` nos filtros esperados — nenhuma regra de negócio (cálculo, arredondamento, RN01–RN12) teve asserção alterada. A suíte de regressão com o catálogo de referência passou sem mudanças.

2. Diagnóstico

2.1 Code smells e violações de padrão

#	Achado	Evidência	Gravidade
D1	**Classe deus no agente**: interpretação do LLM, validação de schema, correspondência com o catálogo (RN09), sugestão e orquestração na mesma classe	`AgenteProjetos.cs` com 374 linhas, 7 responsabilidades	Média
D2	**Sem multi-tenancy**: nenhuma entidade tem `tenantId`; catálogo global; índices únicos globais; usuários do Keycloak sem vínculo a empresa	busca por `tenant` no código: 0 ocorrências	Alta (bloqueia o modelo SaaS)
D3	**Falta de idempotência** em `POST /projetos` e `POST /projetos/{id}/mensagens`	duplo clique ou *retry* cria projeto duplicado e repete a chamada paga ao LLM	Alta
D4	**Sem proteção de uso e custo**: nenhum *rate limit*; o Kong só tinha `jwt` e `cors`	cada mensagem é uma chamada paga	Alta
D5	**Entrada do cliente interpolada em prompts sem tratamento**: termos, nomes e categorias do catálogo entravam com quebras de linha	permite forjar seções ("## Itens atuais...") no prompt	Média
D6	**Saída do LLM sem limites** (quantidade de itens, tamanho de termo, magnitude das medidas)	medidas absurdas chegavam ao cálculo	Média
D7	**Segredos de exemplo aceitos**: `.env.example` traz `troque-esta-senha` e o backend subia com ele	senha pública em produção por esquecimento	Média
D8	**Fronteiras só por convenção**: nada impedia o domínio de referenciar MongoDB, ou o catálogo de depender do agente	sem testes de arquitetura	Média
D9	**Recursos não descartados**: `SemaphoreSlim` em tipo não descartável; `FormUrlEncodedContent` sem `using`	analisador CA1001 e CA2000	Baixa
D10	**Duplicação (DRY)** ao introduzir o tenant em 3 entidades	`TenantId` + `DefinirTenant` repetidos	Baixa (gerada durante a refatoração e corrigida na hora)
D11	**Cliente HTTP do Keycloak sem resiliência** embora `Microsoft.Extensions.Http.Resilience` constasse na stack permitida	nenhum timeout/retry/circuit breaker	Média
D12	**Pacote único do frontend** de 566 kB, com aviso do bundler	`vite build`	Baixa

2.2 Obsolescência

Não há. O projeto já usa .NET 10 (LTS), React 19, TypeScript estrito, MongoDB 8, Keycloak 26 e `npm audit` / `dotnet list package --vulnerable` sem achados. A "modernização" aqui é **arquitetural e de padrões**, não de versão — por isso o item "migração de versão e *records*" do roteiro da Aula 4 foi adaptado: os DTOs de API já eram `record`, e as entidades de domínio são mantidas como classes por causa da mutabilidade controlada (`Projeto`, `Material`) e do mapeamento do driver do MongoDB.

2.3 Itens avaliados e não alterados (com motivo)

Item	Decisão
`catch (Exception)` em `AgenteProjetos` e `CondutorConversa`	**Mantidos.** São fronteiras de falha por desenho (RN12): convertem qualquer erro do LLM/agente na mensagem fixa e no evento SSE de erro, e registram o detalhe no log. Trocar por tipos específicos deixaria vazar erros do SDK para o cliente.
Entidades de domínio como `record`	Mantidas como classe (ver 2.2).
OpenTelemetry (traces e métricas)	**Não feito.** Exige pacotes fora da lista de `tech-stack.md`, o que pede aprovação do Tech Lead. Fica como próxima evolução.
Extrair o agente em um segundo serviço	**Não feito, de propósito** (ADR-014): sem necessidade medida de escala independente, seria complexidade especulativa (YAGNI). As costuras para extrair no futuro foram preparadas.
Ruído dos analisadores `CA2007`, `CA1062`, `CA1515`, `CA1032`	Não aplicados: `ConfigureAwait` é irrelevante em ASP.NET Core, e o *nullable* estrito já cobre as checagens de nulo.

3. Problemas, técnicas e prompts

Cada item abaixo segue o formato **Problema** → **Técnica aplicada** → **Prompt de refatoração** → **Validação**. Os textos completos dos prompts estão em `prompts.md`.

3.1 Blindagem (Fase 0)

- **Problema:** refatorar sem rede de segurança. **Técnica:** *test snapshotting* — rodar a suíte inteira e congelar o baseline (375 testes) antes de qualquer mudança; usar a suíte de regressão com o catálogo de referência como *golden master*. **Validação:** `baseline.md`.

3.2 Multi-tenancy (D2) — ADR-011

- **Problema:** o sistema só atendia uma empresa; para virar SaaS cada empresa precisa de catálogo, usuários e projetos isolados.
- **Técnica aplicada:** *shared collection with discriminator*. `ITenantAtual` lê a claim `tenant_id` do JWT; o filtro do tenant fica **apenas** em `RepositorioMongo` (consultas, substituições e inserção), então nenhum caso de uso precisa lembrar dele. Todas as policies exigem a claim. Índices únicos viram compostos. A gestão de usuários consulta o Keycloak por atributo (`q=tenant_id:…`) e trata usuário de outra empresa como inexistente. Migração idempotente `001-multitenant.js` para bancos existentes.
- **Decisão sênior:** coleção compartilhada num único realm (e não banco ou realm por tenant) — menor custo operacional e sem duplicar a chave fixa do Kong (ADR-010). Registrado como alternativa rejeitada no ADR.
- **Validação:** testes de isolamento dos repositórios, do serviço de usuários e das policies (usuário sem tenant é negado mesmo sendo admin).

3.3 Idempotência (D3) — ADR-012

- **Problema:** repetição de `POST` duplicava projetos e custo de LLM.
- **Técnica aplicada:** *idempotency key* por filtro de endpoint. Guarda a resposta SSE inteira; repetição recebe o mesmo corpo com `Idempotent-Replayed: true`; chave em uso devolve `409`; chave com

outro conteúdo devolve `422`. Falhas transitórias liberam a chave para nova tentativa. Escopo por tenant + usuário; TTL de 24 h. O frontend gera um UUID por ação e o reaproveita após queda de rede.
- **Achado durante os testes:** um teste de impressão digital passou "em falso" (todas as chaves geravam o mesmo resumo) porque o teste não entregava o corpo ao filtro; foi corrigido e o filtro foi confirmado.
- **Descoberto na integração:** o `Idempotency-Key` precisou entrar na lista de cabeçalhos permitidos do CORS do Kong, senão o navegador bloquearia o *preflight*.

3.4 Camada de segurança (D4, D5, D6, D7) — ADR-013

- **Técnica aplicada:** (1) *rate limiting* em duas camadas (Kong por IP; backend encadeado por usuário e por tenant, com cota específica para chamadas ao agente); (2) defesa em profundidade de prompt: sanitização de toda interpolação (`SegurancaDoPrompt.Linha`), limites da saída do LLM e instruções nos prompts versionados de tratar texto como dado; (3) `ValidadorDeSegredos` recusa valores de exemplo na subida.
- **O que já era sólido e foi preservado:** o LLM só tem a tool `buscarMateriais`; cálculo e gravação são do código; `materialId` sugerido é validado contra a lista enviada. Essa é a barreira final contra injeção de prompt.
- **Limitação conhecida:** os limites do backend são por processo (memória); com várias réplicas, o teto efetivo multiplica.

3.5 Desacoplamento do agente (D1) — ADR-014

- **Técnica aplicada:** *Extract Class* / SRP. `AgenteProjetos` virou orquestrador (146 linhas) sobre `IInterpretadorDeDescricao` e `IResolvedorDeMateriais`; JSON e registro de sugestões foram para classes internas compartilhadas.
- **Validação:** os 17 testes originais do agente e a suíte de regressão passaram **sem alterar asserções** — só a construção do agente nos dois pontos de teste mudou.

3.6 Fronteiras verificadas (D8) — ADR-014

- **Técnica aplicada:** testes de arquitetura por reflexão, sem biblioteca nova (respeitando a regra de `tech-stack.md`). Verificam a regra de dependência entre camadas e as fronteiras entre os módulos Catálogo, Cotação e Identidade. Há um teste de guarda contra testes vazios.

3.7 Higiene de código (D9, D10, D11, D12)

- `KeycloakAdminClient` implementa `IDisposable` e descarta `FormUrlEncodedContent` (CA1001, CA2000).
- `EntidadeDoTenant` elimina a duplicação criada ao introduzir o tenant (DRY).
- Pipeline de resiliência no cliente do Keycloak, com retry desligado para métodos não seguros (ADR-015).
- Rotas do frontend em *lazy loading*: pacote principal de 566 kB para 424 kB e fim do aviso do bundler.

4. Antes × depois (código)

Métrica	Antes	Depois	Observação
Linhas de produção (backend)	4.446	5.213	+767: novas capacidades (tenant, idempotência, limites, segurança)
Linhas de testes (backend)	4.179	4.993	+814
Linhas do frontend	2.916	2.961	+45
Testes (backend + frontend)	397	471	+74
`AgenteProjetos.cs`	374	146	−61 %
`catch (Exception)` genéricos	3	3	mantidos por desenho (2.3)
ADRs	10	15	ADR-011 a ADR-015

Esta refatoração **não remove boilerplate** (o projeto já era enxuto): o ganho está em **robustez e sustentabilidade**, e o código de produção cresceu porque a modernização arquitetural adicionou capacidades (isolamento, idempotência, proteção). A legibilidade e a manutenibilidade melhoram onde há redução: o agente, antes monolítico, é agora três classes coesas e testáveis isoladamente.

5. Como validar

```
./scripts/verificar.sh --sem-docker   # build, regressão, testes + cobertura, auditoria de dependências, lint e build do frontend
```

Resultado nesta branch: **todas as verificações passaram** (backend 86 % de cobertura; frontend 84 % de linhas; 0 vulnerabilidades).

O que NÃO foi validado nesta sessão

O ambiente de desenvolvimento não tinha o Docker em execução, então **não foram executados**: a importação do realm do Keycloak (atributo `tenant_id`, mapper da claim, perfil de usuário declarativo), os scripts do MongoDB (`01-colecoes-indices.js` e a migração `001-multitenant.js`, validados apenas quanto à sintaxe) e o `docker compose build`. Antes do merge, rodar o ambiente completo e conferir:

1. Login com `admin.teste` e inspeção do token: deve conter `tenant_id = empresa-demo`.
2. Criar usuário pela tela de usuários e conferir o atributo no console do Keycloak.
3. Duas empresas: criar usuários com tenants diferentes e confirmar que não enxergam catálogo, projetos nem usuários um do outro.
4. Repetir um `POST /projetos` com o mesmo `Idempotency-Key` e conferir `Idempotent-Replayed: true`, sem nova chamada ao LLM.
5. Em banco existente: `./scripts/backup-mongodb.sh` e depois `./scripts/migrar-para-multitenant.sh`.

6. Riscos e pontos de atenção

- **Migração de dados e do realm:** bancos e realms já em uso precisam da migração e da recriação do contêiner do Keycloak (o realm só é importado na criação). Usuários existentes **sem** o atributo `tenant_id` passam a ser negados (`403`) até serem associados a uma empresa. Ver `migracao.md`.

- **Sem administrador de plataforma:** não há perfil que enxergue vários tenants; é uma evolução natural.
- **Idempotência opcional:** clientes que não enviam o cabeçalho continuam sem proteção.
- **`.env` com valores de exemplo agora impede a subida do backend** (intencional).

7. Próximos passos recomendados

1. Executar o roteiro da seção 5 em ambiente com Docker e registrar o resultado.
2. Contador distribuído para os limites de uso (necessário com mais de uma réplica).
3. OpenTelemetry com `correlationId` do navegador até a chamada ao LLM (aprovar os pacotes).
4. Trilha de auditoria (alteração de preço, arquivamento) com *outbox*.
5. Perfil de administrador da plataforma e gestão de tenants.

Prompts utilizados — Refatoração

Entregável da atividade "Refatoração de Arquitetura". Os prompts seguem as três fases do roteiro da Aula 4 (`04.1_Guia_Demonstracao.md`), adaptados ao ProjectPricingAI: o roteiro é voltado a Java (Records, Virtual Threads), e este projeto já é .NET 10, então os prompts de migração de versão foram trocados por prompts de arquitetura e segurança.

> **Nota de transparência.** Os prompts P0.1 e P0.2 são as mensagens do aluno, quase literais. Os prompts P1.1 a P3.2 descrevem, no formato do roteiro, a instrução dada em cada etapa da execução assistida; foram redigidos a partir do que foi feito e são representativos, não uma transcrição palavra a palavra da conversa.

0. Prompts de condução (mensagens do aluno)

P0.1 — Enunciado e ideias do grupo

> Atividade Prática — Refatoração de Arquitetura. Aplicar técnicas de "Modernização de Sistemas" no projeto das aulas anteriores. Identificar dívidas técnicas (inclusive geradas pela própria IA) e realizar uma refatoração estruturada, sem adicionar funcionalidades. Entregáveis: código na branch `refactoring`, prompts utilizados e relatório de modernização. Sugestão do grupo: o sistema está monolítico, sem camada de segurança, sem atenção à idempotência, e os acessos são de usuários de uma única empresa (refatorar para SaaS / tenants diferentes). Pedido: mais ideias de refatoração seguindo o roteiro da Aula 4 e um planejamento. Restrições: não atuar na `main` (criar branch), manter as edições no nome do aluno.

P0.2 — Autonomia nas decisões

> Sobre as decisões minhas e do grupo, tome decisões como um engenheiro sênior especialista. E pode seguir seu plano na branch apartada!

Decisões tomadas a partir de P0.2 (registradas nos ADRs): coleção compartilhada com `tenantId` e tenant em claim do JWT num único realm (ADR-011); monolito modular em vez de extrair o agente (ADR-014); recorte "núcleo + destaques do grupo".

1. Fase de pré-refatoração (blindagem)

P1.1 — Baseline de testes

> Execute a suíte completa do backend do ProjectPricingAI e registre o baseline (quantidade de testes por projeto, tempo, linhas de código de produção e de teste, maiores arquivos, quantidade de `catch (Exception)`). Esse baseline é o *golden master*: nenhum teste existente pode mudar de comportamento durante a refatoração.

P1.2 — Mapeamento de acoplamento e dívida técnica

> Identifique *code smells* arquiteturais no ProjectPricingAI. Verifique como `AgenteProjetos`, `FerramentasCotacao` e os endpoints se acoplam ao domínio e à infraestrutura, se há vazamento de lógica de persistência, e se existe isolamento entre empresas, idempotência e proteção de uso. Rode os analisadores do .NET em modo máximo (`AnalysisMode=All`) e separe sinal de ruído. Liste os riscos de quebrar o sistema em cada refatoração proposta.

2. Fase de execução (transformação assistida)

P2.1 — Multi-tenancy (ADR-011)

> O sistema atende uma única empresa. Refatore para SaaS multi-tenant: o tenant deve vir de uma claim do JWT (nunca do cliente), o isolamento deve ficar num único ponto do código (repositório base) para que nenhum caso de uso possa esquecê-lo, os índices únicos devem passar a valer por tenant e a gestão de usuários do Keycloak deve ser restrita ao tenant do administrador. Compare as alternativas (banco por tenant, realm por tenant, coleção compartilhada), registre a decisão em ADR e escreva a migração dos dados existentes. Atualize os testes apenas onde a assinatura mudou e crie testes de isolamento entre tenants.

P2.2 — Idempotência (ADR-012)

> Torne idempotentes os dois POSTs que acionam o agente (criar projeto e enviar mensagem), cuja resposta é um stream SSE. Use o cabeçalho `Idempotency-Key`, escopo por tenant e usuário, e reenvie a resposta guardada em repetições sem chamar o LLM de novo. Defina o comportamento para chave em andamento, chave com conteúdo diferente e falhas transitórias. Atualize o frontend e o CORS do gateway.

P2.3 — Camada de segurança (ADR-013)

> Adicione uma camada de segurança sem mudar regras de negócio: limites de uso por IP (Kong), por usuário e por tenant (backend), com cota específica para chamadas que acionam o LLM e resposta 429 em Problem Details; defesa contra injeção de prompt (sanitizar tudo que é interpolado em prompts, limitar a saída do LLM, reforçar os prompts versionados); e recusa de segredos com valor de exemplo na subida. Verifique se a modernização não introduziu nova superfície de ataque.

P2.4 — Desacoplamento de domínio e SRP (ADR-014)

> A classe `AgenteProjetos` tem 374 linhas e mistura interpretação do LLM, validação de schema, correspondência com o catálogo e orquestração. Extraia colaboradores com contrato próprio, mantendo o comportamento idêntico: a suíte de regressão e os testes do agente não podem ter nenhuma asserção alterada.

P2.5 — Fronteiras arquiteturais verificáveis (ADR-014)

> Transforme a arquitetura em regra executável: escreva testes que impeçam o domínio de referenciar outras camadas, MongoDB, Anthropic ou ASP.NET, e que impeçam dependências indevidas entre os módulos Catálogo, Cotação e Identidade. Não adicione bibliotecas fora da lista de `tech-stack.md`. Inclua um teste que garanta que o detector não está vazio.

P2.6 — Higiene, DRY e resiliência (ADR-015)

> Corrija os achados reais dos analisadores (CA2000, CA1001), elimine a duplicação que a introdução do tenant criou nas entidades e adicione resiliência (timeout, retry com jitter e circuit breaker) ao cliente HTTP do Keycloak, sem repetir métodos não seguros. Carregue as rotas do frontend sob demanda para reduzir o pacote principal.

3. Fase de pós-refatoração (validação e relatório)

P3.1 — Validação completa

> Rode o script de verificação do projeto (build com avisos como erro, regressão, testes com cobertura mínima de 70 %, auditoria de dependências, lint, formatação e build do frontend) e confirme que todos os 375 testes originais continuam passando.

P3.2 — Relatório de Modernização (dívida paga)

> Gere o comparativo técnico "antes × depois" do ProjectPricingAI: diagnóstico (code smells, violações de padrão, obsolescência), e para cada problema o formato Problema → Técnica aplicada → Prompt → Validação. Inclua o que foi avaliado e **não** alterado, com motivo, e declare explicitamente o que não pôde ser validado no ambiente.

Plano de Refatoração — ProjectPricingAI

> **Status:** executado na branch `refactoring`. Resultado, decisões e o que ficou de fora estão no relatório de modernização; as decisões das seções 9 foram tomadas e registradas nos ADR-011 a ADR-015. Itens executados: Fase 0, A1, A2, A3 (como monolito modular), A4, A5, S1, S2, S3 (CORS), S4 (validação de segredos), S6 (já existia no `verificar.sh`), Q2, Q3 (parcial), Q4, Q5 (já ativo), Q7 (lazy loading), Q8 (Keycloak). Não executados: A6/S5 (auditoria/outbox), Q6 (OpenTelemetry, exige aprovação de pacotes) e S7 (autorização por recurso já concentrada em `ServicoProjetos` e nos repositórios).
>
> Branch: `refactoring` (criada a partir de `main`). Objetivo da atividade: auditoria técnica e modernização, **sem novas funcionalidades de negócio**. Estrutura baseada no roteiro da Aula 4 (Blindagem → Transformação → Validação).

1. Ideias do grupo (ponto de partida)

Ideia	Situação encontrada no código
Monolito	Um único processo `backend` concentra API, agente, MCP, precificação e gestão de usuários. Só há separação lógica em camadas (Dominio/Aplicacao/Infraestrutura/Api).
Sem camada de segurança	JWT é validado no Kong e no backend, mas não há *rate limiting*, limite de tamanho de entrada, proteção contra *prompt injection* nem trilha de auditoria. O `kong.yml` só tem os plugins `jwt` e `cors`.
Idempotência	Não existe nenhuma chave de idempotência. Um `POST /projetos` repetido (duplo clique, retry de rede, reconexão do SSE) cria projetos duplicados e **gasta chamadas pagas à Claude API**.
Multi-tenant (SaaS)	Não existe `tenantId`. Projetos são isolados só por `clienteId`; o catálogo de materiais é global; o realm do Keycloak é único.

2. Princípio de escopo

Refatorar = mudar a estrutura **sem mudar o comportamento observável**. Por isso:

- Tudo o que mudar comportamento (tenant, idempotência, rate limit) entra como **"modernização arquitetural"**, com decisão registrada em ADR, e não como refatoração pura.
- O que for refatoração pura (DRY, SOLID, dead code, records, exceções) é protegido pelos testes existentes (~4,2 mil linhas) + suíte de regressão `casos-referencia.json`.
- O projeto já está em .NET 10 / React 19 / Mongo 8, portanto **não há migração de versão**. A "modernização" aqui é arquitetural e de padrões de código.

3. Fase 0 — Blindagem (antes de mexer em código)

#	Ação	Entregável
0.1	Rodar `scripts/verificar.sh` e registrar baseline (testes passando, cobertura, tempo de build).	`docs/refactoring/baseline.md`
0.2	Medir métricas "antes": linhas por projeto, maiores arquivos, complexidade ciclomática, nº de `catch (Exception)`, avisos do analisador.	tabela "Antes" do relatório
0.3	**Testes de caracterização** para lacunas: fluxo de `CondutorConversa`/SSE, concorrência otimista, isolamento por `clienteId`, regras de arredondamento (RN08).	novos testes verdes **antes** das mudanças
0.4	**Auditoria de dívida técnica** (prompt de mapeamento de acoplamento) gerando a lista priorizada.	`docs/refactoring/auditoria.md`

Prompt-base (adaptado do roteiro): "*Identifique code smells arquiteturais no ProjectPricingAI. Verifique como `AgenteProjetos`, `FerramentasCotacao` e os endpoints se acoplam ao domínio e à infraestrutura. Liste os riscos de quebrar o sistema em cada refatoração proposta.*"

4. Fase 1 — Transformação (itens priorizados)

Legenda: **Tipo** R = refatoração pura · M = modernização arquitetural (muda comportamento, exige ADR) · Esforço P/M/G.

4.1 Eixo Arquitetura

ID	Item	Tipo	Esforço	Justificativa / técnica
A1	**Multi-tenancy (SaaS)**: `TenantId` em `Material`, `Projeto`, `Conversa`; resolução do tenant por claim do JWT; filtro global nos repositórios (`RepositorioMongo`); índices únicos de `materiais.nome/sinonimos` passam a ser compostos com `tenantId`. Estratégia de isolamento: coleção compartilhada com discriminador (decisão em ADR-011, comparando com banco por tenant e realm por tenant).	M	G	Atende a sugestão do grupo. Isolamento forçado na camada de repositório evita vazamento entre tenants por esquecimento.
A2	**Idempotência**: cabeçalho `Idempotency-Key` em `POST /projetos` e `POST /projetos/{id}/mensagens`; coleção `chavesIdempotencia` com TTL; resposta repetida devolve o resultado salvo sem chamar o LLM de novo.	M	M	Evita duplicidade e custo de LLM. Já existe `Versao` (concorrência otimista) para reaproveitar.
A3	**Monolito → monolito modular** com fronteiras explícitas: módulos `Catalogo`, `Cotacao`, `Identidade`, cada um com contrato público e testes de arquitetura (NetArchTest/ArchUnitNET) proibindo dependência cruzada.	R	G	Passo realista para o "monolito": modularizar primeiro e deixar a extração do agente em serviço como opção documentada em ADR, em vez de microsserviços prematuros (YAGNI).

ID	Item	Tipo	Esforço	Justificativa / técnica
A4	**Desacoplar o agente do provedor de LLM**: extrair `AgenteProjetos` (374 linhas, maior arquivo) em colaboradores pequenos: `InterpretadorDescricao`, `ClassificadorCorrespondencias`, `SugestorMaterial`, `MontadorResposta`.	R	M	SRP/SOLID; torna cada etapa testável sem mock do `IChatClient` inteiro.
A5	**Domain Services puros**: garantir que regra (RN03, RN05, RN09) não dependa de repositório; `FerramentasCotacao` vira só orquestrador.	R	M	Equivale ao prompt "Refatoração de Domínio (Desacoplamento)" do roteiro.
A6	**Eventos de domínio / outbox** para auditoria (projeto cotado, material alterado).	M	M	Base da trilha de auditoria (S5) sem acoplar ao fluxo principal. Opcional.

4.2 Eixo Segurança

ID	Item	Tipo	Esforço	Justificativa
S1	**Rate limiting** por usuário/tenant no backend (`AddRateLimiter`) e no Kong (`rate-limiting`), mais cota diária de chamadas ao LLM por tenant.	M	P	Protege custo da Claude API (hoje ilimitado).
S2	**Defesa contra prompt injection**: delimitar a entrada do cliente no prompt, limitar tamanho da descrição, validar que `materialId` e termos vindos do LLM pertencem ao tenant (já há validação contra a lista; estender). Testes com payloads maliciosos.	M	M	Risco específico de sistemas com LLM; relevante para o relatório.
S3	**Cabeçalhos e CORS**: revisar a lista de origens do Kong, CSP no Nginx do frontend, `HSTS` condicionado a HTTPS.	R	P	Hardening barato.
S4	**Segredos**: remover padrões "troque-esta-senha" funcionais, validar na subida que nenhum segredo está com valor de exemplo; preparar abstração `ISegredos` para o AWS Secrets Manager previsto no `tech-stack.md`.	R	P	Dívida declarada na própria documentação.
S5	**Trilha de auditoria** (quem alterou preço de material, quem arquivou projeto).	M	M	Exigência típica de SaaS; combina com A6.
S6	**Auditoria de dependências**: `dotnet list package --vulnerable` e `npm audit` no `verificar.sh`.	R	P	Fecha o item de "vulnerabilidades" do roteiro.
S7	**Autorização por recurso** (verificar `tenantId` + `clienteId` num único ponto, não espalhado nos endpoints).	R	M	Elimina risco de IDOR.

4.3 Eixo Qualidade de código (técnicas do roteiro)

ID	Item	Técnica
Q1	Trocar `catch (Exception)` genéricos (`AgenteProjetos.cs:83`, `CondutorConversa.cs:49`) por hierarquia semântica e tratamento centralizado (`IExceptionHandler`/Problem Details já existe em `TratadorDeExcecoes`; consolidar).	Tratamento de exceções

ID	Item	Técnica
Q2	Eliminar duplicação (DRY) entre `MaterialRepository`, `ProjetoRepository` e `ConversaRepository` e entre validadores.	DRY
Q3	Converter DTOs/requisições/respostas e *value objects* em `record` imutáveis; usar `required` / `init`; coleções `IReadOnlyList` / *collection expressions* C# 14.	Modernização sintática
Q4	Substituir *magic strings* de códigos de erro e nomes de coleções por constantes/enums; remover código morto (usar `dotnet format` + analisadores IDE0051/IDE0052/CA1822).	Dead code / KISS
Q5	Habilitar `Nullable` estrito + `TreatWarningsAsErrors` + `AnalysisMode=All` e zerar os avisos.	Padronização
Q6	Observabilidade: OpenTelemetry (traces + métricas), `correlationId` do front ao LLM, métricas de tokens e latência por etapa do agente.	Modernização
Q7	Frontend: separar componentes grandes (`PaginaCotacao`, `useConversa`), tipar eventos SSE com *discriminated unions* e `zod`, tratamento de erro único.	SRP / tipagem
Q8	Resiliência no cliente do LLM: *retry* com *jitter*, *circuit breaker* e *timeout* via `Microsoft.Extensions.Http.Resilience` (já na stack).	Resiliência

5. Fase 2 — Validação e evolução

#	Ação
5.1	Reexecutar `verificar.sh` e a suíte de regressão; **nenhum teste de caracterização pode ser alterado** sem justificativa escrita.
5.2	Testes novos por item: isolamento entre tenants (A1), repetição da mesma chave (A2), 429 (S1), payloads de injeção (S2), testes de arquitetura (A3).
5.3	Revisão de segurança dos pontos novos (prompt de auditoria de injeção e IDOR).
5.4	Coletar métricas "depois" e gerar o **Relatório de Modernização — Dívida Paga** (antes × depois).

6. Ordem sugerida e recorte

Não é necessário fazer tudo. Recomendação de recorte para a entrega, equilibrando valor e risco:

1. **Núcleo (obrigatório):** Fase 0 completa, Q1, Q2, Q3, Q4, Q5, A4, A5.
2. **Destaques que o grupo pediu:** A1 (tenant), A2 (idempotência), S1 + S2 (camada de segurança), A3 (modularização em vez de microsserviços).
3. **Se sobrar tempo:** S4, S5/A6, Q6, Q8, Q7.

Cada item = um commit (ou PR pequeno) com mensagem `refactor(<área>): ...`, testes verdes antes e depois.

7. Riscos e cuidados

- **A1 é o item mais invasivo:** mexe em índices do Mongo, Keycloak (claim `tenantId` por usuário/organização) e em todos os repositórios. Fazer com migração de dados idempotente e tenant padrão para dados existentes.

- A resposta SSE e o agente dependem da Claude API: **nenhum teste novo deve chamar a API real** (manter mock do `IChatClient`).
- Mudanças de comportamento (M) precisam constar no relatório como *modernização*, não como refatoração pura, para manter a honestidade metodológica.
- Registrar **todos os prompts** usados em `docs/refactoring/prompts.md` (entregável da atividade).

8. Entregáveis finais

Entregável	Local
Código refatorado	branch `refactoring`
Prompts utilizados	`docs/refactoring/prompts.md`
Relatório de Modernização (Diagnóstico → Problema / Técnica / Prompt, mais antes × depois)	`docs/refactoring/relatorio-modernizacao.md`
ADRs novas (tenant, idempotência, modularização)	`.ai/architecture.md`

9. Decisões pendentes do grupo

1. Estratégia de tenant: coleção compartilhada com `tenantId` (recomendada) ou banco por tenant?
2. O `tenantId` virá de claim no Keycloak (atributo/organização) em um único realm?
3. Entregar A3 como monolito modular (recomendado) ou extrair o agente em um segundo serviço?